



External Design Document v1 for Help Desk Ticket Management System



CENTENNIAL COLLEGE

PROGRESSCAMPUS



COMP229003.91783.2025F

Web Application Development (SEC. 003)

Professor: Julio Azevedo de Carvalho

Project First Release: Help Desk Ticket Management System

PRESENTED BY: LEADGLOBE

Ayoola Praise 301461719

Ean Bynoe 301390261

Moesha Aurelle Emaleu Deutou 301444365

Jan Robi Mistica 301360862

Yeshon Suresh Kumar 301404005

Zuhriddin Sharofiddinov 301454665



External Design Document v1 for Help Desk

Ticket Management System

Table of Contents

Project Overview	3
System Features.....	3
Use Case Diagram	3
System Architecture Diagram	4
Database Schema Diagram ERD.....	5
Wireframes (Mockups).....	6
Login Page.....	6
Register Page	8
Dashboard Page	9
Ticket Creation Form	10
Ticket Details (Admin View).....	11
Technology Stack	11
API Testing and Validation (Postman Results)	12
References	17



External Design Document v1 for Help Desk Ticket Management System

Project Overview

The Help Desk Ticket Management System (HD-TMS) is a full-stack web application designed to streamline customer support and technical issue tracking. The system allows registered users to create and view tickets for issues, while administrators can update, monitor, and close tickets through an interactive dashboard.

The backend uses Node.js, Express, and MongoDB, providing secure REST APIs with JWT authentication. The frontend (React or EJS) will connect to the API to display dashboards, forms, and status views.

System Features

Feature Area	Description	Actors
User Management	Register, login, logout, profile editing, role-based access (User/Admin).	User, Admin
Site Security	JWT-based authentication and authorization for route protection.	System
Ticket Dashboard	Displays all open tickets in a list with filter options to view closed tickets.	User, Admin
Ticket Creation	Form to create new tickets with description, priority, and customer info.	User
Ticket Management	Admins can update ticket status, add comments, and resolutions.	Admin
Audit Trail	Ticket “iterations” log status changes, comments, and timestamps.	System
Database	MongoDB stores Users and Tickets with relationships.	—

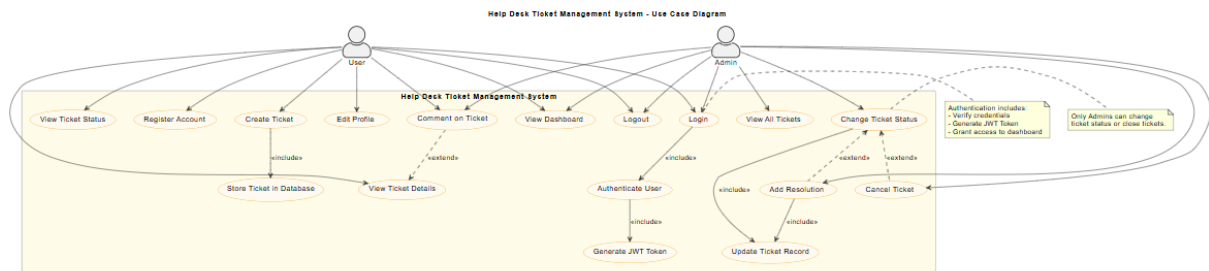
Use Case Diagram

The Use Case Diagram above illustrates all the major interactions within the Help Desk Ticket Management System. Regular users can register, log in, and manage tickets, while



External Design Document v1 for Help Desk Ticket Management System

administrators can update, resolve, or cancel tickets. Authentication and ticket persistence are handled internally by the system using secure JWT-based APIs connected to MongoDB Atlas.

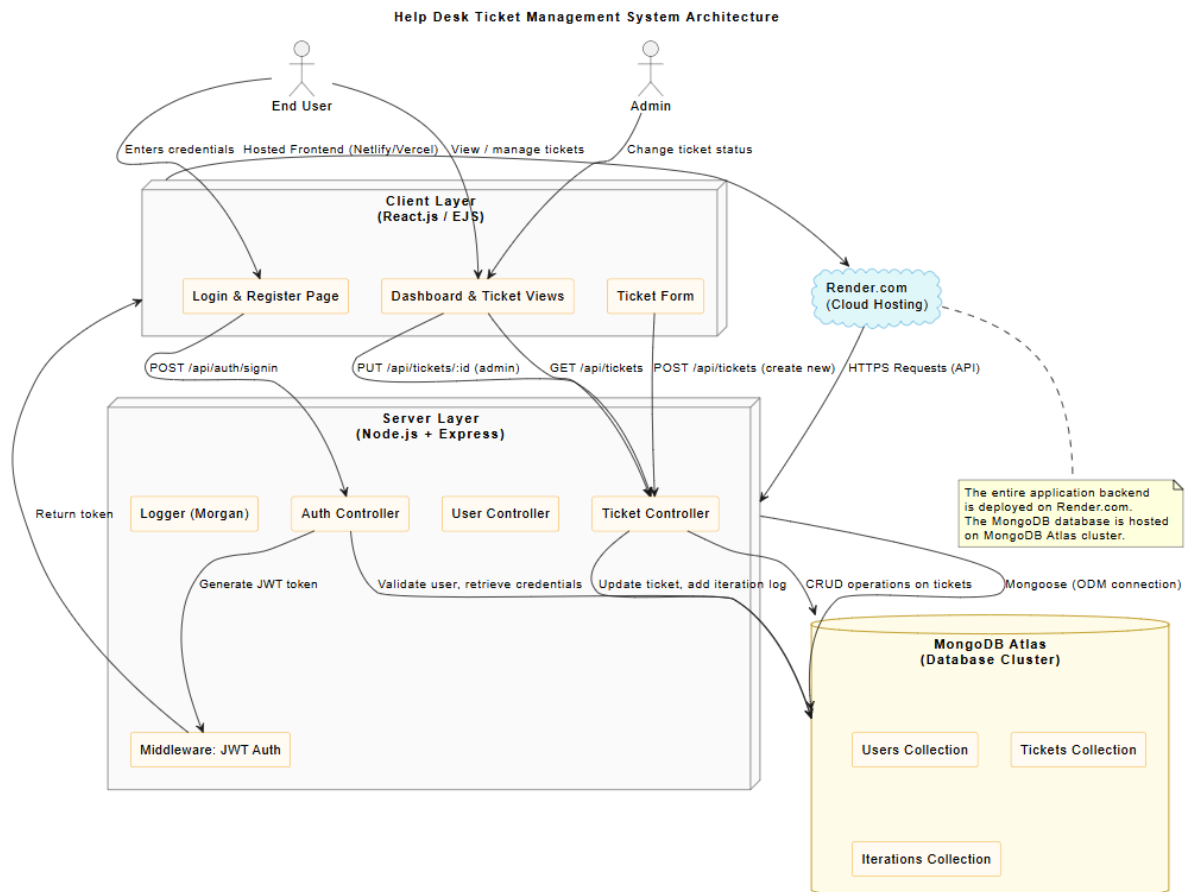


System Architecture Diagram

The architecture above illustrates the layered structure of the Help Desk Ticket Management System. Users and Admins interact through the frontend (React/EJS) which communicates with an Express.js backend via RESTful APIs. The backend enforces security using JWT authentication and stores data persistently in MongoDB Atlas. The system is deployed on Render.com for cloud access and scalability.



External Design Document v1 for Help Desk Ticket Management System

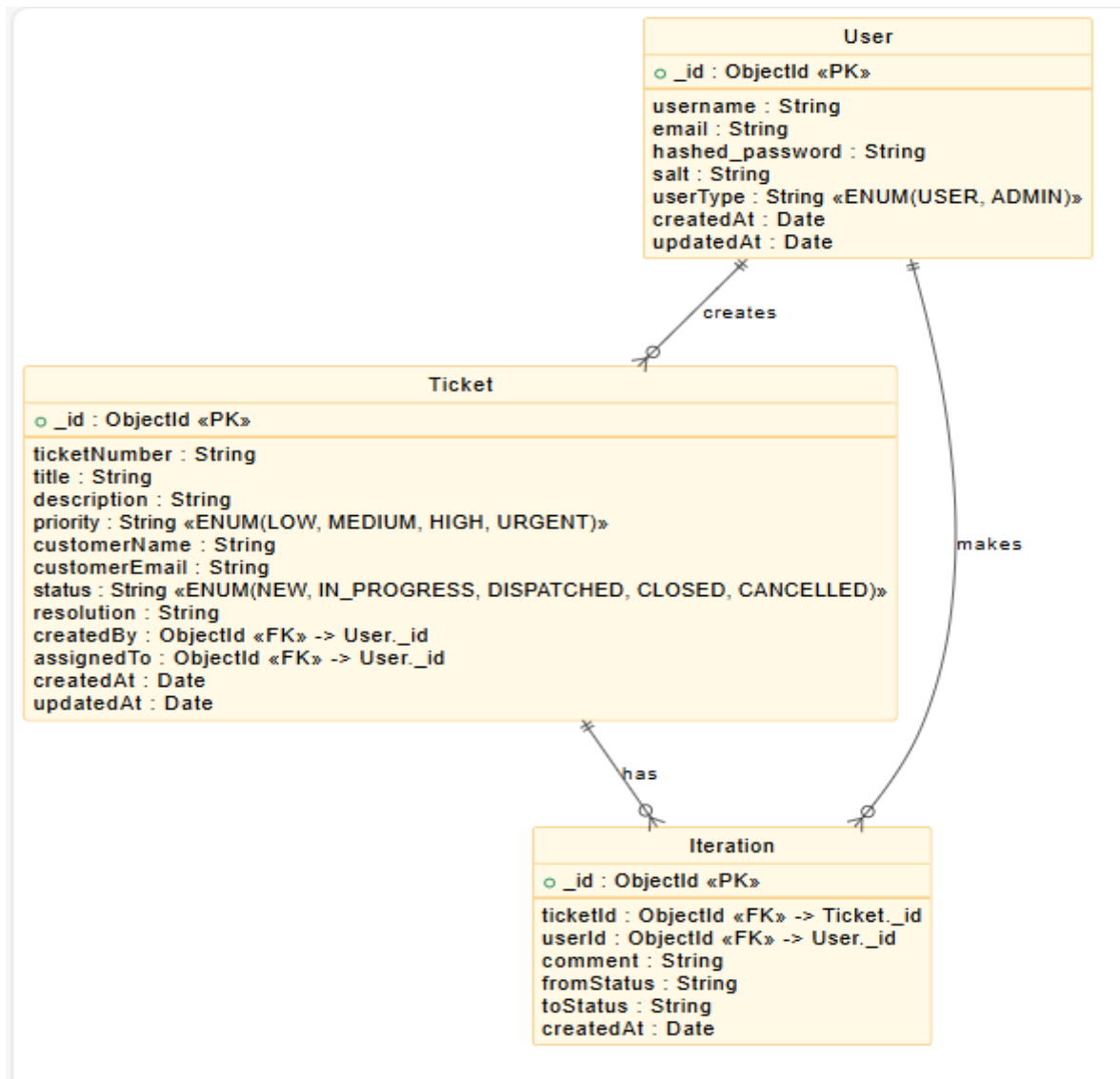


Database Schema Diagram ERD

The diagram illustrates the logical data model used in the Help Desk application. It defines three main entities — User, Ticket, and Iteration — and their relationships. Each User can create multiple Tickets, and each Ticket can have multiple Iterations that record comments, status changes, and timestamps, providing a complete audit trail. The diagram also specifies primary and foreign keys, data types, and ENUM values for fields such as *userType*, *priority*, and *status*. This structure ensures referential integrity, supports efficient data retrieval via Mongoose in MongoDB, and aligns with the backend's Express MVC implementation.



External Design Document v1 for Help Desk Ticket Management System



Wireframes (Mockups)

The following wireframes represent the core user interface screens for the LEADGLOBE Help Desk Portal.

These pages illustrate both User and Admin interactions, covering login, registration, ticket management, and administrative updates. Each screen has been designed for clarity, usability, and consistency with the backend MVC structure.

Login Page



External Design Document v1 for Help Desk Ticket Management System

LOGIN PAGE

LEADGLOBE
Help Desk Portal

Authentication

Email Address:

Password:

[SIGN IN](#) [REGISTER](#)

[Forgot Password?](#)

Message: "Invalid credentials" (if login fails)

Description:

The user enters their credentials and clicks Login. If authentication fails, an *"Invalid credentials"* message appears. The Register button navigates to the account creation page.



External Design Document v1 for Help Desk Ticket Management System

Register Page

A mockup of the registration page. It features a light green background with a rounded rectangular form in the center. The form has a light gray background and contains the following elements: the "LEADGLOBE" logo in pink, the text "Register New Account" in black, four input fields (Email Address, Password, Confirm Pass, and User Type) with pink borders, and a "Sign Up" button at the bottom. The User Type field is a dropdown menu with "User" and "Admin" options. The Password and Confirm Pass fields show masked text (xxxxxxxxxx).

REGISTRATION PAGE

LEADGLOBE
Register New Account

Email Address:

Password:

Confirm Pass:

User Type: User ▼ Admin ▼

Sign Up

Description:

Users fill out the form to create a new account.

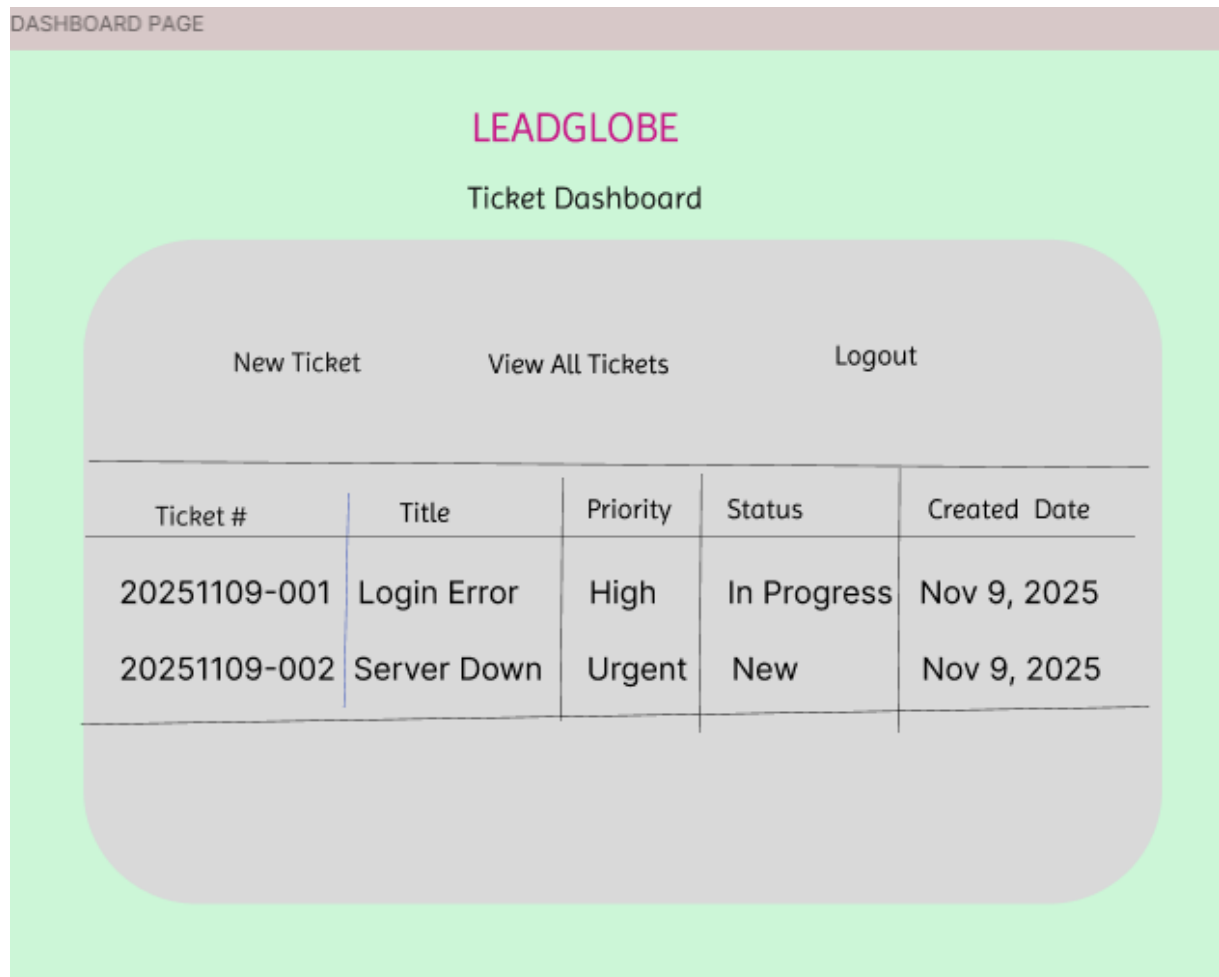
Upon successful registration, their data is stored in MongoDB (Users collection).

Admins are assigned different privileges through the *userType* field.



External Design Document v1 for Help Desk Ticket Management System

Dashboard Page



Description:

Users can view their open tickets.

Admins see all tickets in the system.

Clicking a ticket opens the Ticket Details page.

Closed tickets are hidden by default unless "View All Tickets" is selected.



External Design Document v1 for Help Desk Ticket Management System

Ticket Creation Form

TICKET CrEATION FORM

LEADGLOBE

Create New Ticket

Title:

Description:

Priority: Low ▼ Medium ▼ High ▼ Urgent ▼

Customer Name:

Customer Email:

Submit Ticket



External Design Document v1 for Help Desk Ticket Management System

Ticket Details (Admin View)

Ticket Details (Admin View)

LEADGLOBE

Admin - Ticket Details

Admin - Ticket Details

Ticket #: 20251109-002

Customer Name: John Smith

Customer Email: john.smith@example.com

Status: New ▼ In Progress ▼ Dispatched ▼ Closed ▼ Cancelled ▼

Resolution:

Comment:

Update Ticket

Description:

Admin users can modify the Status, Resolution, or add a Comment.

Once the status is set to *Closed*, the ticket becomes read-only to maintain an audit trail.

All updates are recorded in the *Iterations* collection for history tracking.

Technology Stack

Layer	Technology	Purpose
Frontend	React.js / EJS + Bootstrap	User interface
Backend	Node.js + Express	Server logic & routing
Database	MongoDB Atlas + Mongoose	Data storage



External Design Document v1 for Help Desk Ticket Management System

Layer	Technology	Purpose
Auth	JWT (JSON Web Tokens)	Secure authentication
Dev Tools	Postman, Visual Studio Code	Testing & development
Hosting	Render / Railway	Cloud API deployment
Version Control	GitHub	Source code management
Agile Tracking	Jira	Task tracking and backlog

API Testing and Validation (Postman Results)

This section presents the **Postman test results** for the Help Desk Ticket Management System APIs. Each screenshot demonstrates that the backend API endpoints are functioning correctly with proper request and response handling. All tests were conducted using **Node.js**, **Express**, and **MongoDB Atlas** connections.

Tested Endpoints

Feature	HTTP Method	Route	Description
Create User	POST	/api/users/	Registers a new user in the system
Get All Users	GET	/api/users/	Fetches all registered users
Delete User	DELETE	/api/users/:id	Deletes a specific user by ID
Delete All Users	DELETE	/api/users/	Deletes all users (for testing purposes)
Authenticate User	POST	/api/Auth/SignIn	Validates user credentials and returns a JWT token

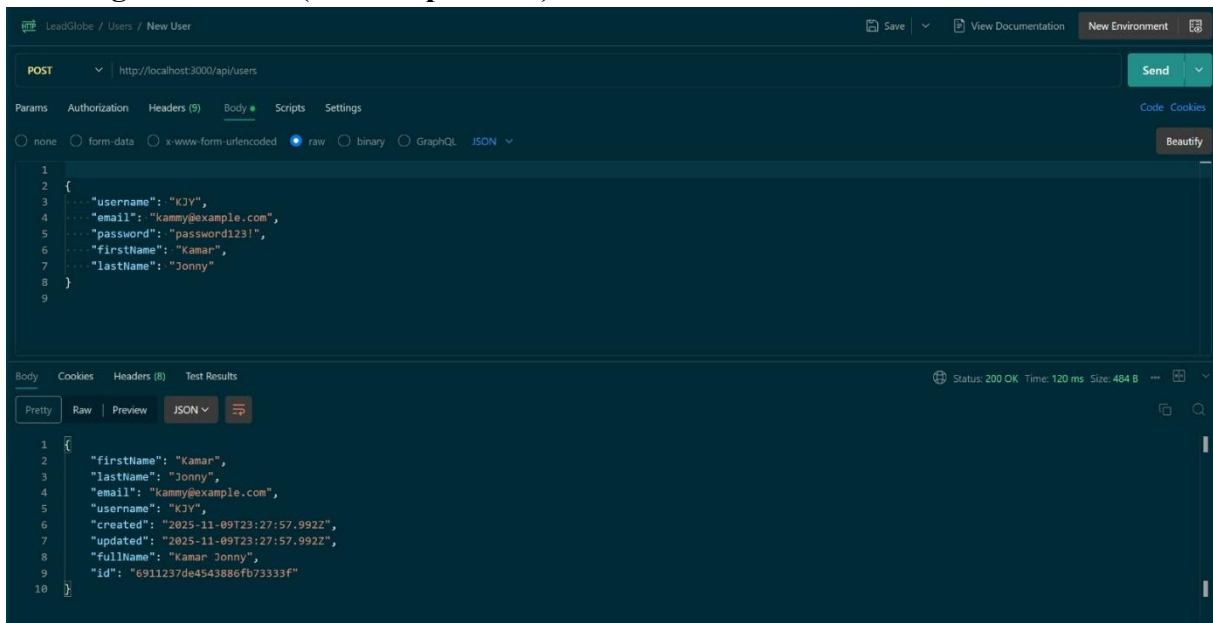


External Design Document v1 for Help Desk Ticket Management System

Feature	HTTP Method	Route	Description
Create Event	POST	/api/events/	Adds a new event entry to the database
Get All Events	GET	/api/events/	Retrieves all created events

Screenshots

Creating a New User (POST /api/users/)





External Design Document v1 for Help Desk Ticket Management System

Getting All Users (GET /api/users/)

LeadGlobe / Users / GET All

GET | http://localhost:3000/api/users/

Params Authorization Headers (7) Body Scripts Settings

Query Params

Key	Value
Key	Value

Body Cookies Headers (8) Test Results

Status: 200 OK Time: 124 ms Size: 1.56 KB

Pretty Raw Preview JSON

```
1 [{"firstName": "Deangelo", "lastName": "Jones", "email": "dj@example.com", "username": "DJones", "created": "2025-11-09T21:56:47.450Z", "updated": "2025-11-09T21:56:47.451Z", "fullName": "Deangelo Jones", "id": "69110e1fe02b3626d7e38bc"}, {"firstName": "Kai", "lastName": "Griffin", "email": "kaib@example.com", "username": "KGB", "created": "2025-11-09T22:21:36.608Z", "updated": "2025-11-09T22:21:36.608Z", "fullName": "Kai Griffin", "id": "691113f0e02b3626d7e38cc"}, {"firstName": "Nahtan", "lastName": "Ruck", "email": "nate@example.com", "username": "NateRuck", "created": "2025-11-09T22:22:33.760Z", "updated": "2025-11-09T22:22:33.760Z", "fullName": "Nahtan Ruck", "id": "69111429e02b3626d7e38d2"}, {"firstName": "Test", "lastName": "User", "email": "test2@example.com", "username": "testuser2", "created": "2025-11-09T22:24:26.394Z", "updated": "2025-11-09T22:24:26.394Z", "fullName": "Test User", "id": "6911149ae02b3626d7e38db"}, {"firstName": "Aren", "lastName": "Brathwaite", "email": "Baren@example.com", "username": "ArenB", "created": "2025-11-09T22:27:02.037Z", "updated": "2025-11-09T22:27:02.037Z", "fullName": "Aren Brathwaite", "id": "6911153671e4344aaf6071c5"}, {"firstName": "Kamar", "lastName": "Jonny", "email": "kammy@example.com", "username": "KJY", "created": "2025-11-09T23:27:57.992Z", "updated": "2025-11-09T23:27:57.992Z", "fullName": "Kamar Jonny", "id": "6911237de4543886fb73333f"}]
```

Deleting a User (DELETE /api/users/:id)

LeadGlobe / GET by id

GET | http://localhost:3000/api/users/69110834e02b3626d7e38b4

Params Authorization Headers (7) Body Scripts Settings

Query Params

Key	Value
Key	Value

Body Cookies Headers (8) Test Results

Status: 200 OK Time: 97 ms Size: 490 B

Pretty Raw Preview JSON

```
1 {"firstName": "Deangelo",
2 "lastName": "Jones",
3 "email": "dj@example.com",
4 "username": "DJones",
5 "created": "2025-11-09T21:31:32.203Z",
6 "updated": "2025-11-09T21:31:32.204Z",
7 "fullName": "Deangelo Jones",
8 "id": "69110834e02b3626d7e38b4"
9 }
```



HTTP

LeadGlobe / Users / deleteALL

Save

View Documentation

New Environment

DELETE

http://localhost:3000/api/users/

Send

Params

Authorization

Headers (7)

Body

Scripts

Settings

Code

Cookies

Query Params

Key	Value
Key	Value

Body

Cookies

Headers (8)

Test Results

Status: 200 OK Time: 168 ms Size: 323 B

Pretty

Raw

Preview

JSON

```
1 {
2   "success": true,
3   "message": "Users deleted successfully."
4 }
```

LeadGlobe / Authentication / signin

POST http://localhost:3000/api/auth/signin

Params Authorization Headers (9) Body **Scripts** Settings

none form-data x-www-form-urlencoded **raw** binary GraphQL JSON

```
1 {
2   "email": "kammy@example.com",
3   "password": "password123!"
4 }
```

Body Cookies Headers (8) Test Results

Pretty Raw Preview **JSON**

```
1 [{"success":true,"message":"Authentication successful","token":"eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY5MTYyHj1kZTQ1NDM4ODZyIjczMzMBZiIsInVzZXQyVWllIjo1S8pZi1iawF0IjoXNzYyNzNzNzYzLzC1eHh0IjE3Nj13Mz1Nj9.YLdbQZcAaytwEB-LLX_177Uq9Lsy6-YgtNkMasuCGLaqvRbvVP3zRPh0I6mxuP1oskegPewhSmpkP2nL08L8g"}]
```

Status: 200 OK Time: 162 ms Size: 570 B



External Design Document v1 for Help Desk Ticket Management System

Creating an Event (POST /api/events/)

The screenshot displays a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:3000/api/events
- Body (raw):**

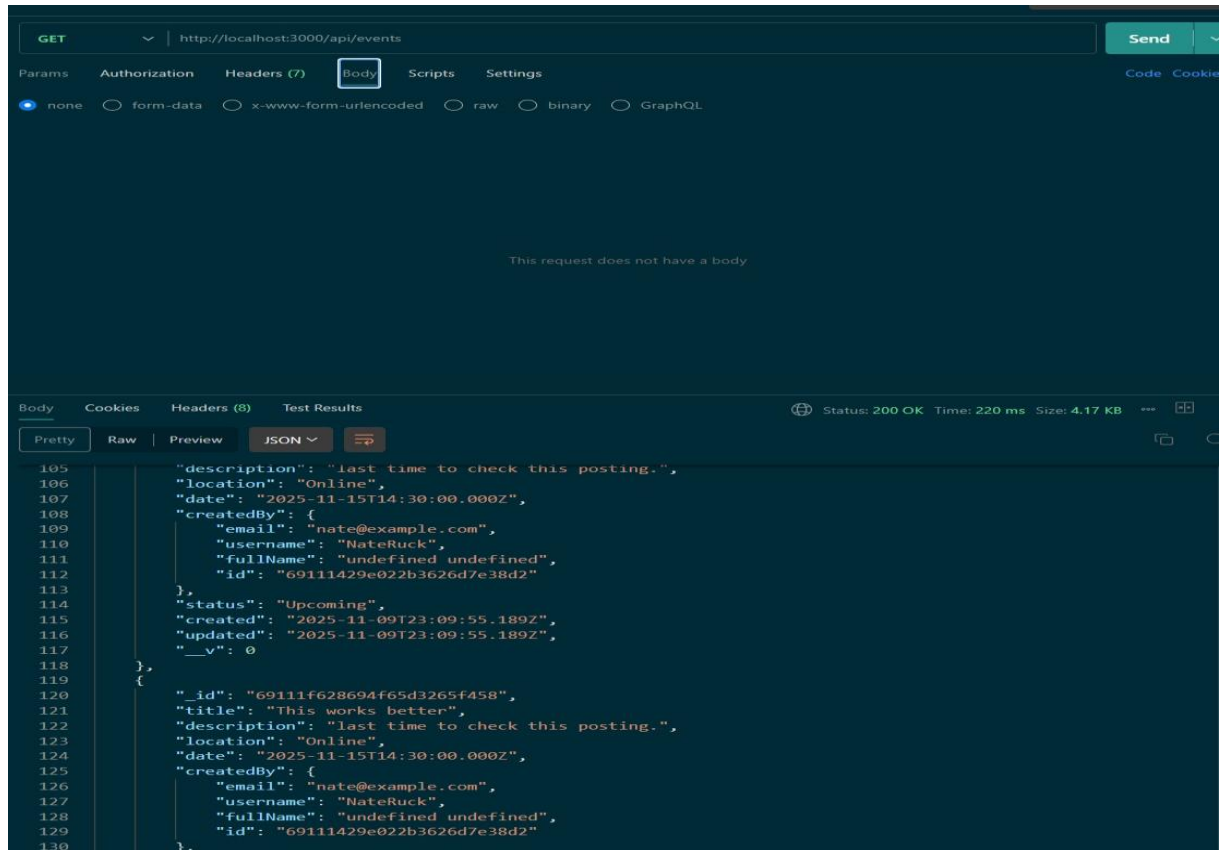
```
1 {
2   "title": "This works better",
3   "description": "last time to check this posting.",
4   "location": "Zoom",
5   "date": "2025-11-15T14:30:00.000Z",
6   "createdBy": "6911269de4543886fb73334f"
7 }
```
- Status:** 200 OK, Time: 960 ms, Size: 637 B
- Response Body (JSON):**

```
1 {
2   "success": true,
3   "message": "Event created successfully.",
4   "data": {
5     "title": "This works better",
6     "description": "last time to check this posting.",
7     "location": "Zoom",
8     "date": "2025-11-15T14:30:00.000Z",
9     "createdBy": "6911269de4543886fb73334f",
10    "status": "Upcoming",
11    "_id": "691129dbe4543886fb733357",
12    "created": "2025-11-09T23:55:07.349Z",
13    "updated": "2025-11-09T23:55:07.349Z",
14    "__v": 0
15  }
16 }
```




External Design Document v1 for Help Desk Ticket Management System

Getting All Events (GET /api/events/)



References

Deployed project in render: <https://project-part-2-vbxc.onrender.com>

GitHub: <https://github.com/Zuck7/project-part-2>